ShanghaiTech University

CS101 Algorithms and Data Structures
Fall 2024

Homework 11

Wenye Xiong 2023533141

Due date: December 22, 2024, at 23:59

1. Please write your solutions in English.
2. Submit your solutions to Gradescope.
3. Set your FULL name to your Chinese name and your STUDENT ID correctly in Gradescope account settings.
4. If you want to submit a handwritten version, scan it clearly. `CamScanner` is recommended.
5. We recommend you to write in LaTeX.
6. When submitting, match your solutions to the problems correctly.
7. No late submission will be accepted.
8. Violations to any of the above may result in zero points.

Demand of the Algorithm Design: (MUST READ!)

All of your algorithms should need the three-part solution, this will help us to score your algorithm. You should include **main idea, proof of correctness and run-time analysis.** The detail is as below:

1. The **main idea** of your algorithm. You should correctly convey the idea of the algorithm in this part. It does not need to give all the details of your solutions or explain why they are correct. When grading these problems, we will emphasize how you define your sub-problems, whether your Bellman equation is correct, and the correctness of your complexity analysis:
   (a) **Define your subproblems clearly.** Your definition should include the variables you choose for each subproblem and a brief description of your subproblem in terms of the chosen variables.
   (b) Your **Bellman equation** should be a recurrence relation whose **base case** is well-defined.
   (c) You can **briefly explain each term in the equation** if necessary, which might improve the readability of your solution and help us grade it.
2. A **proof of correctness**. You must prove that your algorithm work correctly, no matter what input is chosen. For iterative or recursive algorithms, often a useful approach is to find an invariant. A loop invariant needs to satisfy three properties: (1) it must be true before the first iteration of the loop; (2) if it is true before the ith iteration of the loop, it must be true before the $i + 1$st iteration of the loop; (3) if it is true after the last iteration of the loop, it must follow that the output of your algorithm is correct. You need to prove each of these three properties holds. Most importantly, you must specify your invariant precisely and clearly. **However, sometimes the problem does not require you to prove the correctness of your algorithm. But remember to give your bellman Equation clearly.** If you invoke an algorithm that was proven correct in class, you don't need to re-prove its correctness.
3. The asymptotic **running time** of your algorithm, stated using $\Theta(\cdot)$ notation. And you should have your **running time analysis**, i.e., the justification for why your algorithm's running time is as you claimed. Often this can be stated in a few sentences (e.g.: "the loop performs $|E|$ iterations; in each iteration, we do $\Theta(1)$ Find and Union operations; each Find and Union operation takes $\Theta(\log |V|)$ time; so the total running time is $\Theta(|E| \log |V|)$"). Alternatively, this might involve showing a recurrence that characterizes the algorithm's running time and then solving the recurrence.
4. You only need to calculate the optimal value in each problem of this homework and you don't have to back-track to find the optimal solution.

1. (0 points) (DP Example) Maximum Subarray Problem

Given an array $A = \langle A_1, \ldots, A_n \rangle$ of n elements, please design a dynamic programming algorithm to find a contiguous subarray whose sum is maximum.

Solution:

Main Idea

Let $OPT(I)$ be the maximum sum of subarrays of A ending with A_i. Then we have the following equation:

$$OPT(i) = \begin{cases} A_1 & \text{if } i = 1 \\ \max\{A_i, A_i + OPT(i-1)\} & \text{otherwise} \end{cases}$$

The final answer is

$$\max_{i \in \{1,2,\ldots,n\}} OPT(i)$$

Proof of Correctness:

- The 1st term in max: only take A_i
- The 2nd term in max: take A_i together with the best subarray ending with A_{i-1}

Runtime Analysis:

The running time is $\Theta(n)$ since we have a single loop with range n to reduce subproblems.

2. (8 points) Odd number of coins changing

Given n coin denominations $\{c_1, c_2, \cdots, c_n\}$ and a target value V, you are going to make change for V. However, only odd number of coins is allowed.

Please design a **dynamic programming** algorithm that find the fewest odd number of coins needed to make change for V (or report impossible).

Hints: *For this problem, you can formulate your subproblems as:*

$$F(v) = \text{fewest odd number of coins to make change for } v.$$
$$G(v) = \text{fewest even number of coins to make change for } v.$$

Solution:

Main Idea

Let $F(v)$ be the fewest odd number of coins to make change for v and $G(v)$ be the fewest even number of coins to make change for v. Then we have the following equations:

$$F(v) = \begin{cases} 0 & \text{if } v = 0 \\ \min_{i=1}^{v \geq c_i} \{1 + G(v - c_i)\} & \text{if } v > 0 \end{cases}$$

$$G(v) = \begin{cases} 0 & \text{if } v = 0 \\ \min_{i=1}^{v \geq c_i} \{1 + F(v - c_i)\} & \text{if } v > 0 \end{cases}$$

The final answer is $F(V)$ if it exists, otherwise report impossible.

Proof of Correctness:

- Base case: When $v = 0$, no coins are needed
- For $v > 0$: To get odd number of coins for v, we take one coin c_i and combine it with even number of coins for remaining value $v - c_i$
- Similarly for $G(v)$: Take one coin and combine with odd number of coins for remaining value
- The minimum across all coin choices gives optimal solution

Runtime Analysis:

- For each value from 1 to V, we consider n coins
- We need to compute both F and G arrays
- Total runtime: $O(nV)$
- Space complexity: $O(V)$

3. (10 points) Happiness Planning

In the magical world of Hogwarts, Professor Slughorn likes to plan his potions classes with precision and flair.

For the next m months, starting with no Galleons, Slughorn will brew potions and earn x Galleons per month. During the i-th month ($1 \leq i \leq m$), there will be a singular chance to spend c_i Galleons to acquire happiness worth h_i. However, it's a fair bargain for Slughorn to buy happiness, that is, the amount of c_i will not be too big.

Borrowing is not permitted. Galleons earned in the i-th month can only be used in a subsequent j-th month ($j > i$).

Since potion masters don't dabble with Muggle technology, please help him design a dynamic programming algorithm to assist Slughorn in discovering the maximum achievable sum of happiness.

Hints: *Still consider the backpack problem! What should you design the Bellman equation with limitation?*

Solution:

Main Idea

Let $OPT(i, j)$ be the maximum happiness achievable after i months with j Galleons. Then we have the following equation:

$$OPT(i,j) = \begin{cases} 0 & \text{if } i = 0 \\ \max \begin{cases} OPT(i-1, j-x) & \text{not buy in month i} \\ OPT(i-1, j-x-c_i) + h_i & \text{buy in month i if } j - x \geq c_i \end{cases} & \text{otherwise} \end{cases}$$

The final answer is

$$\max_j OPT(m, j)$$

Proof of Correctness:

- Base case: At month 0, no happiness can be achieved
- For each month i and current Galleons j:
  - Option 1: Don't buy happiness, just accumulate x Galleons
  - Option 2: Buy happiness if we have enough Galleons ($j - x \geq c_i$)
- The maximum happiness at month m with any amount of Galleons is our answer

Runtime Analysis:

Let G be the maximum possible Galleons ($G = mx$). Time complexity: $O(mG)$ since we have two nested loops:

- Outer loop: m months
- Inner loop: G possible Galleon values

Space complexity: $O(mG)$ to store the DP table.

4. (10 points) Greedy doesn't work

Tom and Jerry are playing an interesting game, where there are n cards in a line. All cards are faced-up and the number on every card is between 2-9. Tom and Jerry take turns. In anyone's turn, they can take one card from either the right end or the left end of the line. The goal for each player is to maximize the sum of the cards they have collected.

(a) (3') Tom decides to use a greedy strategy: "On my turn, I will take the larger of the two cards available to me." Show a small counterexample ($n \leq 5$) that Tom will lose if he plays this greedy strategy. Assuming Tom goes first and Jerry plays optimally. Tom could have won if he also played optimally.

Solution: Consider the sequence: [2, 4, 9, 3] and Tom goes first.
Tom's greedy strategy: Tom takes 3, Jerry takes 9, Tom takes 4, Jerry takes 2
Tom's score: 3+4=7, Jerry's score: 9+2=11
Tom loses, but Tom could have won by taking 2 first. Then Jerry take 4, Tom take 9, Jerry take 3. Tom's score: 2+9=11, Jerry's score: 4+3=7

(b) (7') Jerry decides to use dynamic programming to find an algorithm to maximize his score, assuming he is playing against Tom and Tom is using the greedy strategy from part (a). Help Jerry to develop the dynamic programming solution. The solution you proposed should not worse than $O(n^2)$ in time complexity.

Solution:

Main Idea

Let $OPT(l, r)$ be Jerry's maximum score when the remaining cards are from index l to r. Then we have the following equation:

$$OPT(l,r) = \begin{cases} 0 & \text{if } l \geq r \\ \begin{cases} \max\{A_{l+1} + OPT(l+2, r), A_r + OPT(l+1, r-1)\} & \text{if } A_l > A_r \\ \max\{A_l + OPT(l+1, r-1), A_{r-1} + OPT(l, r-2)\} & \text{if } A_r > A_l \end{cases} & \text{otherwise} \end{cases}$$

The final answer is $OPT(1, n)$

Proof of Correctness:

- Base case: When no cards remain or only one card remains, score is 0. Because Tom goes first, Jerry can pick no card.
- For each state (l,r):
  - If $A_l > A_r$: Tom takes A_l, Jerry can take from A_{l+1} or A_r
  - If $A_r > A_l$: Tom takes A_r, Jerry can take from A_l or A_{r-1}
  - Jerry chooses the maximum score he can get.

- After both players take cards, recursively solve remaining subproblem

Runtime Analysis:

- Time complexity: $O(n^2)$ since we have $O(n^2)$ states (l,r)
- Space complexity: $O(n^2)$ to store the DP table

5. (10 points) Pairwise DNA Sequence Alignment

Given two DNA sequences: a query sequence $Q = \langle Q_1, \ldots, Q_m \rangle$ of m nucleotides and a subject sequence $S = \langle S_1, \ldots, S_n \rangle$ of n nucleotides. There are 4 types of nucleotides, namely Adenine (A), Cytosine (C), Guanine (G) and Thymine (T). We provide a scoring matrix for nucleotides at the same index of these two sequences:

	A	C	G	T
A	1	-5	-1	-5
C	-5	1	-5	-1
G	-1	-5	1	-5
T	-5	-1	-5	1

where $\text{Score}(X, X)$ on the diagonal represents the score of a successful match for nucleotide X at the same index of both sequences, and $\text{Score}(X, Y)$ indicates the penalty for mismatching nucleotide X by nucleotide Y.

For example, assume we have $m = n$ here and there are two sequences $Q = \langle TGGTG \rangle$ and $S = \langle ATCGT \rangle$. Then the alignment score for these two sequences is

$$\begin{aligned}
\text{Score}(Q, S) &= \sum_{i}^{n} \text{Score}(Q_i, S_i) \\
&= \text{Score}(T, A) + \text{Score}(G, T) + \text{Score}(G, C) + \text{Score}(T, G) + \text{Score}(G, T) \\
&= (-5) + (-5) + (-5) + (-5) + (-5) \\
&= -25
\end{aligned}$$

However, if $m \neq n$, we must insert several gaps '$-$' to make two sequences the same length. In order to align two sequences for higher scores, we can also insert an arbitrary number of gaps '$-$' to each sequence at an arbitrary index. However, adding one gap will result in a gap penalty $\text{Penalty}(X, -) = \text{Penalty}(-, Y) = -2$.

Assume after inserting 4 gaps, we obtain $Q' = \langle -T - GGTG \rangle$ and $S' = \langle ATCG - T- \rangle$. Then the recomputed alignment score is:

$$\begin{aligned}
\text{Score}(Q', S') &= \text{Penalty}(-, A) + \text{Score}(T, T) + \text{Penalty}(-, C) + \text{Score}(G, G) \\
&\quad + \text{Penalty}(G, -) + \text{Score}(T, T) + \text{Penalty}(G, -) \\
&= (-2) + 1 + (-2) + 1 + (-2) + 1 + (-2) \\
&= -5
\end{aligned}$$

Notice that Q' and S' should share the same length after inserting gaps.

Given the scoring matrix and gap penalty, please come up with a dynamic programming algorithm to **maximize the pairwise alignment score** of a pair of DNA sequences by inserting gaps into these two sequences.

(a) (2') Define your subproblem for this question.

Solution: We can define the subproblem as follows: $OPT(i, j)$ is the maximum alignment score of the prefix of Q of length i and the prefix of S of length j. i and j for $0 \leq i \leq m$ and $0 \leq j \leq n$.

(b) (6') Give your Bellman equation to solve the subproblems. (Proof of correctness is not required)

Solution: We have defined the subproblem as $OPT(i, j)$, the maximum alignment score of the prefix of Q of length i and the prefix of S of length j. Then we have the following equation:

$$OPT(i,j) = \begin{cases} Penalty(X, -) * max(i, j) & \text{if } i = 0 \text{ or } j = 0 \\ \max \begin{cases} OPT(i-1, j-1) + \text{Score}(Q_i, S_j) \\ OPT(i-1, j) + \text{Penalty}(Q_i, -) \\ OPT(i, j-1) + \text{Penalty}(-, S_j) \end{cases} & \text{otherwise} \end{cases}$$

(c) (1') What is the answer to this question in terms of the subproblems?

Solution: The answer to this question is $OPT(m, n)$, where m and n are the lengths of the two sequences.

(d) (1') What is the runtime complexity of your algorithm?

Solution: The runtime complexity of the algorithm is $O(mn)$, since we have mn subproblems to solve and each subproblem takes constant time.

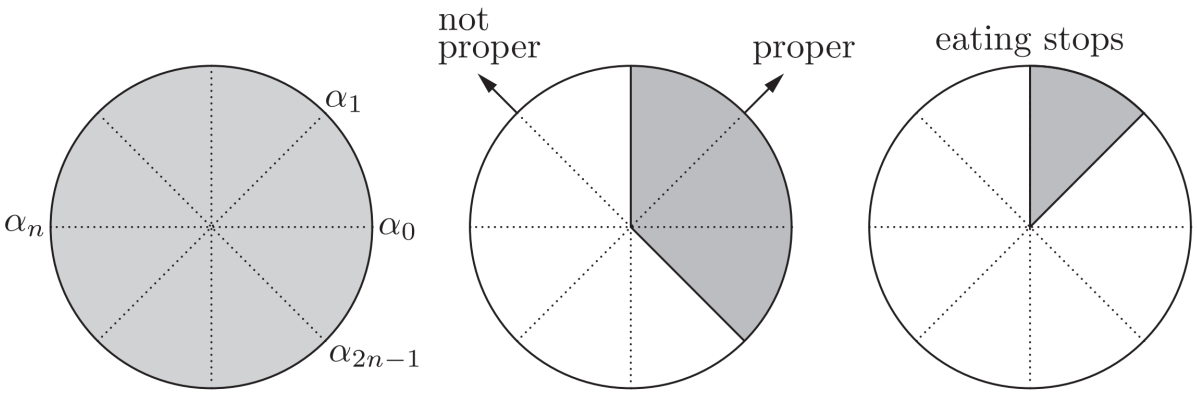
6. (4 points) Pizza Partitioning

Logan and his good friend Joel want to share a round pizza pie that has been cut into $2n$ equal sector slices along rays from the center at angles $\alpha_i = \frac{i\pi}{n}$ for $i \in \{0, 1, ..., 2n\}$, where $\alpha_0 = \alpha_{2n}$. Each slice i between angles α_i and α_{i+1} has a known integer tastiness t_i (which might be negative). To be "fair" to her friend, Logan decides to eat slices in the following way:

- They will each take turns choosing slices of pizza to eat: Logan starts as the chooser.
- If there is only one slice remaining, the chooser eats that slice, and eating stops.
- Otherwise the chooser does the following:
  - Angle α_i is proper if there is at least one uneaten slice on either side of the line passing through the center of the pizza at angle α_i.
  - The chooser picks any number $i \in \{1, ..., 2n\}$ where α_i is proper, and eats all uneaten slices counter-clockwise around the pizza from angle α_i to angle $\alpha_i + \pi$.

Logan wants to maximize the total tastiness of the slices she will eat. Describe an $O(n^3)$-time dynamic programming algorithm to find the maximum total tastiness Logan can guarantee herself via this selection process.

Hints: *As they eat pizza, the uneaten slices are always cyclically consecutive. That is, each time the chooser eats all uneaten slices within half side, There is no chance that two consecutively parts of pizza are left uneaten. You can choose consecutive cyclic subarrays as the subproblems.*



(a) (2') As a kickoff, let $v(i, j)$ be the tastiness of the j slices counter-clockwise from angle α_i. Write the formula over $v(i, j)$.

Solution: $v(i,j) = \sum_{k=0}^{j-1} t_{((i+k) \mod 2n)}$ for $i \in \{0, 1, ..., 2n-1\}$ and $j \in \{0, 1, 2, ..., n\}$

(b) (2') Define your subproblem for this question.

Solution: Define $dp(i, j, p)$ as the maximum total tastiness Logan can guarantee herself with j slices counter-clockwise from angle α_i and p indicating whether it is Logan's turn to choose.(0 for Logan, 1 for Joel)
for $i \in \{0, 1, ..., 2n-1\}$, $j \in \{0, 1, 2, ..., n\}$ and $p \in \{0, 1\}$

(c) (4' (bonus)) Give your Bellman equation to solve the subproblems. (Proof of correctness is not required)

Solution: The Bellman equation is as follows:

$$dp(i, j, 0) = \max_{k \in \{1,...,j-1\}} \{\max(v(i, k) + dp(i + k, j - k, 1), v(i + k, j - k) + dp(i, k, 1))\}$$

$$dp(i, j, 1) = \min_{k \in \{1,...,j-1\}} \{\min(dp(i + k, j - k, 0), dp(i, k, 0))\}$$

for the base case, $dp(i, 1, 0) = v(i, 1)$ and $dp(i, 1, 1) = 0$ for $i \in \{0, 1, ..., 2n-1\}$

(d) (2' (bonus)) What is the answer to this question in terms of the subproblems?

Solution: The answer is $\max_{i \in \{0,1,...,2n-1\}} \{dp(i, n, 1) + v((i + n) \mod 2n, n)\}$

(e) (2' (bonus)) Give an explanation and time complexity analysis of this algorithm.

Solution:

Explanation: The total tastiness of all slices is a constant number, so while Logan is maximizing her tastiness, Joel is minimizing his tastiness. We define the subproblem as the maximum total tastiness Logan can guarantee herself with j slices counter-clockwise from angle α_i and p indicating whether it is Logan's turn to choose. From the Hints, we know that the uneaten slices are always cyclically consecutive. So we can choose consecutive cyclic subarrays as the subproblems. We can use the Bellman equation to solve the subproblems. The answer is the maximum total tastiness Logan can guarantee herself.

For each subproblem, chooser can choose any proper angle, for any $k \in \{1, ..., j-1\}$, the angle α_{i+k} is proper. And chooser can either eat the slices counter-clockwise from angle α_i to α_{i+k} or from α_{i+k} to α_{i+j}. This is because we only require the line passing through the center of the pizza at angle α_i to have at least one uneaten slice on either side. So both choices are valid. So we need to consider all possible choices of k and choose the maximum (minimum) tastiness.

Time Complexity: For the initialization of getting $v(i, j)$, it takes $O(n^3)$ time.

For the Bellman equation, we need to consider all possible choices of k for each subproblem, which takes $O(n)$ time. We have $O(n^2)$ subproblems. So the total time complexity is $O(n^3)$. Also, we have to compute the final answer, which takes $O(n)$ time.

So the total time complexity is $O(n^3)$.